SOFTWARE ENGINEERING LAB · PROJECT-BASED LEARNING

Unified Student Placement System

Job Posting & Eligibility Verification · Job Application & Tracking

Vineet Shah · Roll No. 28 · 5S CSE, A6-B2
Software Engineering Lab (24CS01PR0502) · Prof. D. A. Borikar

2 modules built

155 test cases

49 Jira work items

Why campus placement needs one system

1 Manual screening

Placement officers check every student against every company's CGPA, branch and backlog rules by hand.

2 Errors and missed deadlines

Ineligible and duplicate applications, stale copies of data in spreadsheets, late applications.

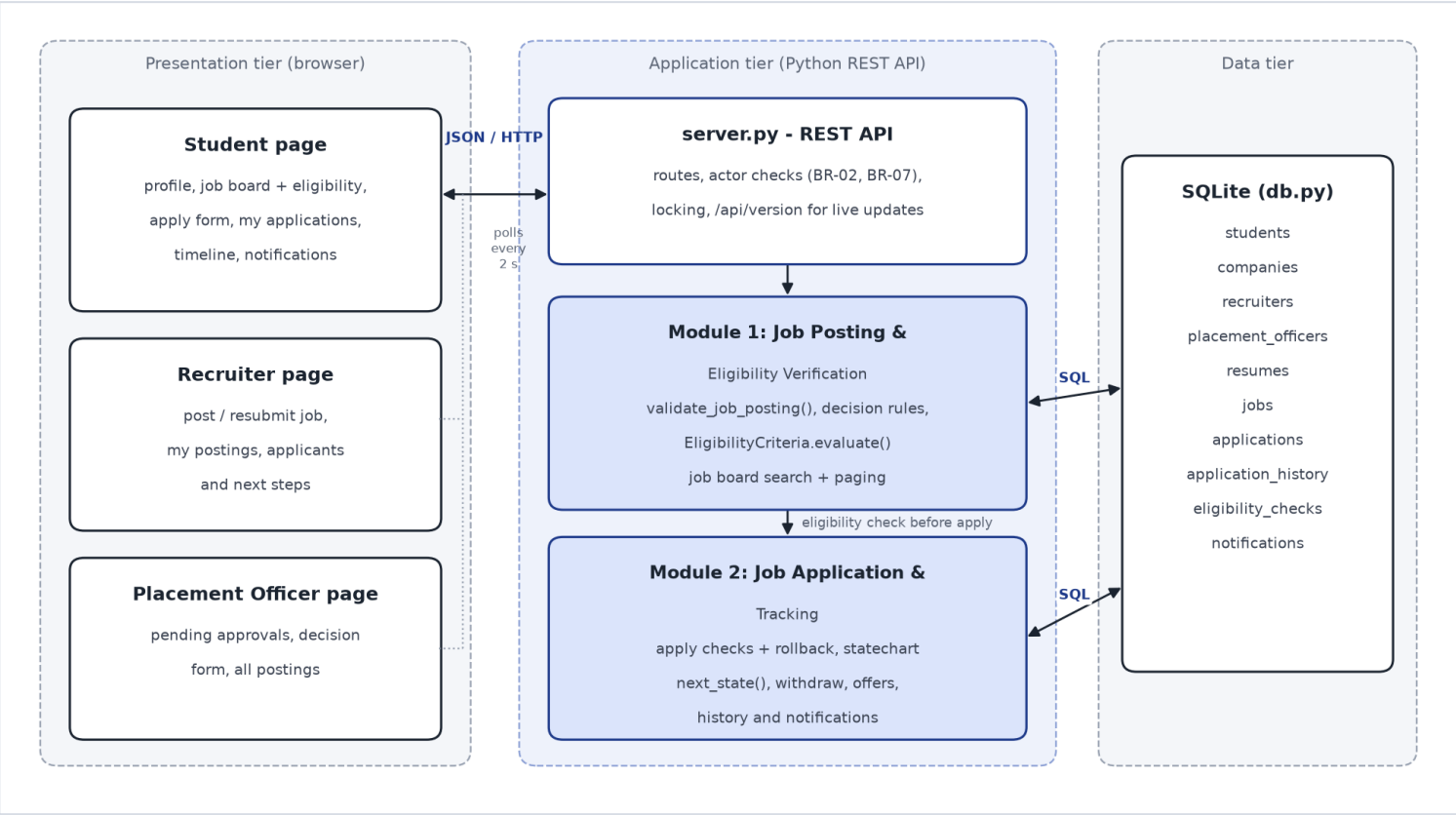
3 No visibility

Students and recruiters cannot see where an application stands; updates go out by e-mail and notice boards.

USPS (my SE lab SRS)

One web platform for students, recruiters and the placement cell: approved job postings, automatic eligibility with reasons, applications with tracking ids and a live status timeline.

What this PBL builds



Implemented

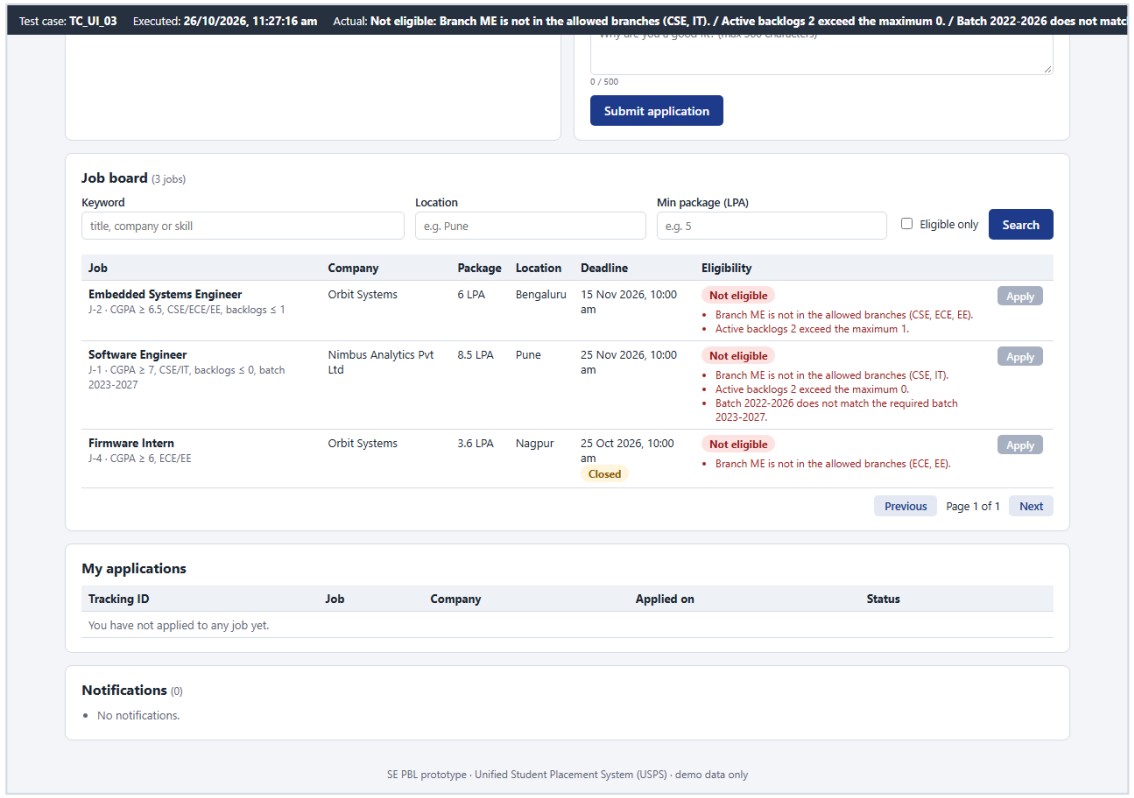
Module 1 - job posting, approval, search, eligibility check
Module 2 - application, statechart workflow, tracking, notifications

Built from my lab work

SRS FR-09 to FR-17, use cases UC-1 / UC-2, Expt-3 class diagram, Expt-5 statechart, SRS 6.1 tables

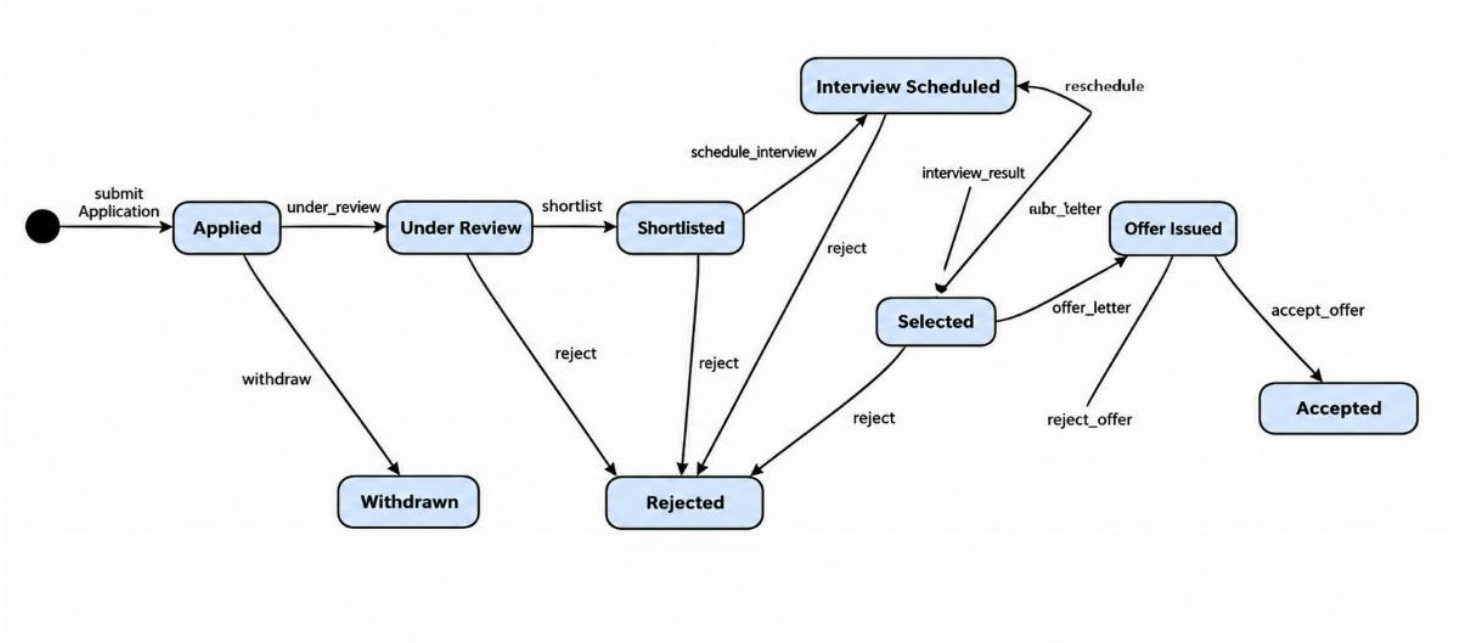
Python 3.11 standard library · SQLite · HTML, CSS, JavaScript

Module 1 · Job posting and eligibility verification



1. **Post a job (FR-10)**
   title, package, location, deadline + eligibility rule-set; validated on the server
2. **Company approval (FR-09)**
   recruiter of an unapproved company cannot post
3. **Officer decision (FR-11, BR-02)**
   approve, reject or return with remarks; returned jobs are resubmitted
4. **Search and filter (FR-12)**
   keyword, location, minimum package, eligible only, pages
5. **Eligibility check (FR-14, BR-03)**
   every unmet criterion listed; Apply disabled when not eligible

Module 2 · Job application and tracking



Apply (UC-1)

published, before deadline, not placed, eligible, no duplicate, own active resume; tracking id USPS-26-0001; saved in one transaction

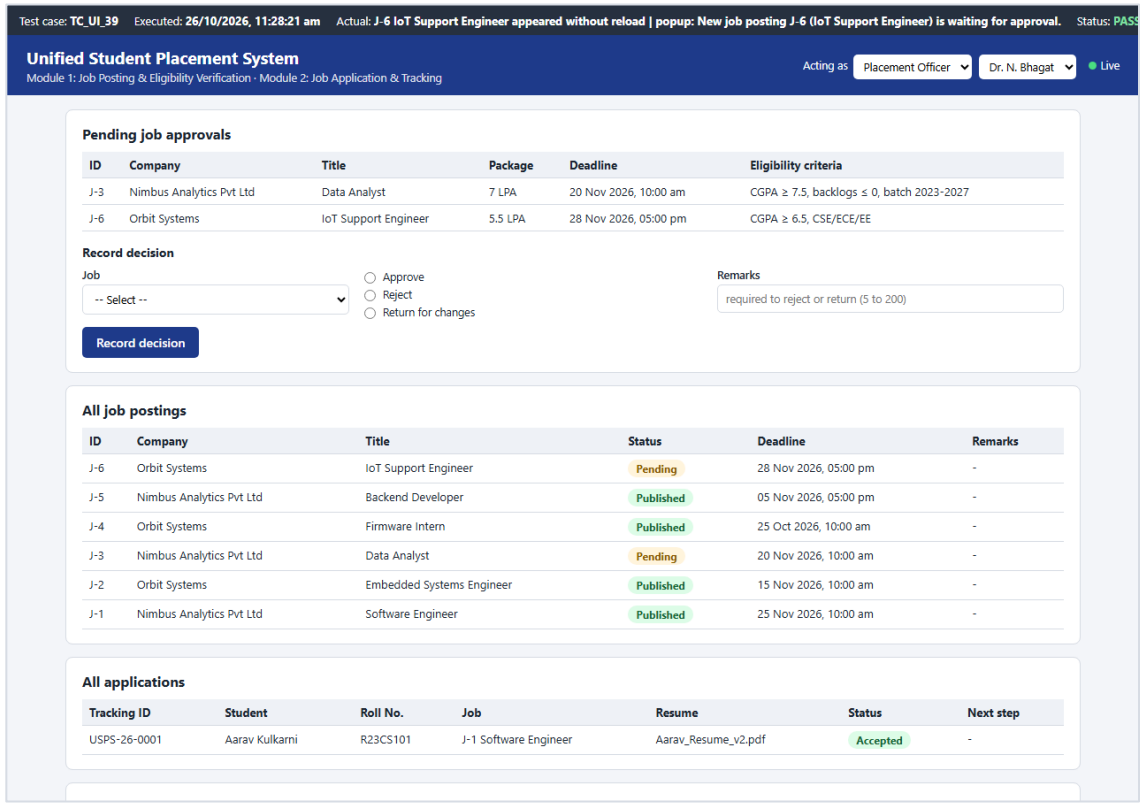
Statechart (Expt-5)

every step goes through next_state(); offers only after selection (BR-08); recruiters touch only their own applicants (BR-07)

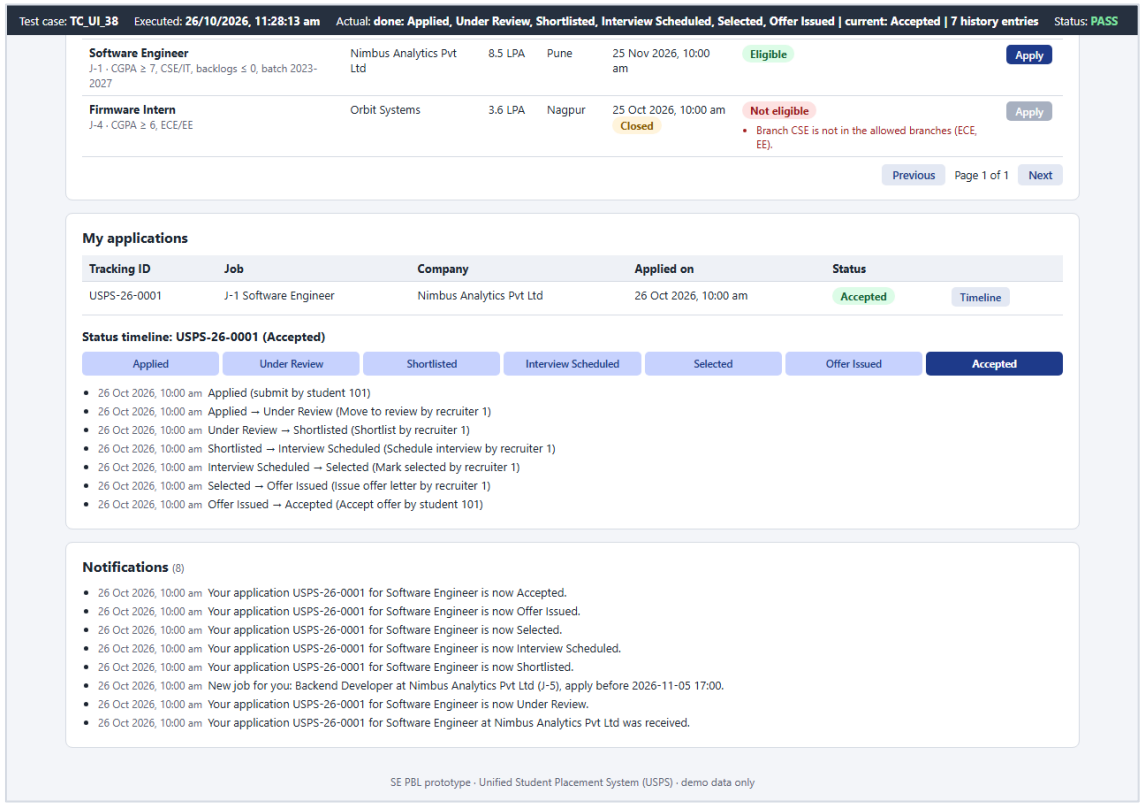
Tracking (FR-17)

timeline with every step and actor, notifications, withdraw before the deadline, accept marks the student placed (BR-01)

Three roles, one live page



Officer: a job posted by a recruiter elsewhere appears without reload, with a pop-up

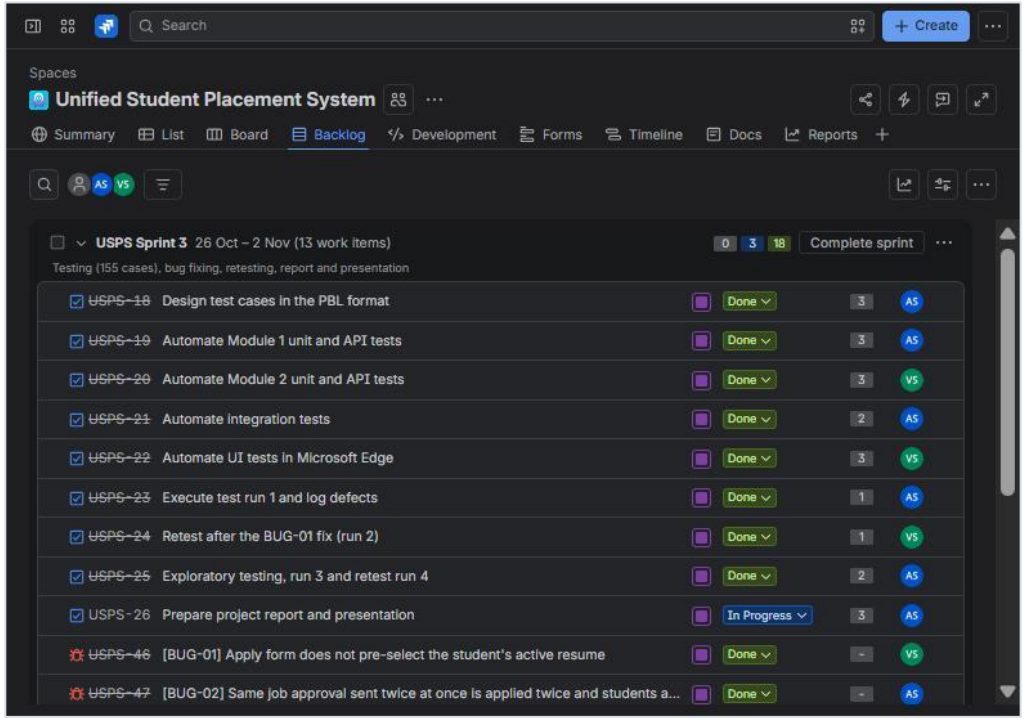


Student: status timeline after accepting the offer

Every open page asks /api/version every 2 seconds and redraws itself when anyone changes data

Managed in Jira · space USPS

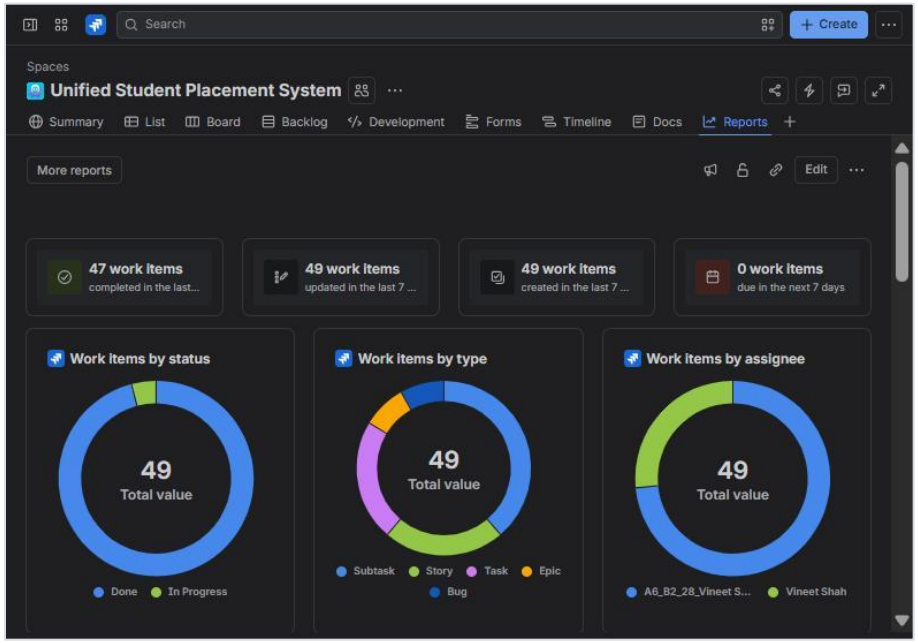
4 epics	**11** user stories	**11** tasks
19 sub-tasks	**4** bugs	**3** sprints



- Epic -> Story / Task / Bug -> Sub-task, with story points and priorities
- Workflow: To Do -> In Progress -> In Review -> Done
- Two members: my institute ID (admin) and my second ID, invited, with 14 work items assigned
- Each bug has steps, severity, priority and a fix + retest comment

Three sprints, tracked to Done

Sprint	Goal	Items	Points	State
USPS Sprint 1 **26-10 10:30-10:50**	Requirements and Module 1: job posting, approval, search and eligibility check	7	22	Completed
USPS Sprint 2 **26-10 10:50-11:05**	Module 2: application, statechart workflow, tracking, notifications and live updates	6	21	Completed
USPS Sprint 3 **26-10 - 02-11**	Testing (155 cases), bug fixing, retesting, report and presentation	13	21	Active



Sprint 3 holds the testing tasks and the four bugs; the report and presentation task stays In Progress until submission.

47 of 49 work items Done on 26-10-2026.

155 automated test cases

61	42	11	41
Module 1 unit + API	Module 2 unit + API	Integration end to end	UI tests in Edge

✓ **Functional**	every requirement checked through the API and the page
✓ **Equivalence partitioning**	valid / invalid classes: words for numbers, unknown branch, wrong resume
✓ **Boundary value analysis**	CGPA 6.99 / 7.00, cover note 500 / 501, apply 1 min before / at deadline
✓ **Validation**	every API and form error message
✓ **Unit and integration**	rules alone, then both modules + DB: rollback, concurrency, restart

Format: Test Case ID · Module · Test Scenario · Test Data · Expected Result · Actual Result · Status

Testing found 4 real bugs

136 / 151

Run 1: passed

151 / 151

Run 2 retest: passed

152 / 155

Run 3 +4 cases: passed

155 / 155

Run 4 retest: passed

- Run 1: 15 UI failures, one root cause (BUG-01)
- Exploratory testing after run 2 found BUG-02 to BUG-04; new cases written
- Every bug logged in Jira, fixed, retested

BUG-01 · USPS-46 — Major / High

Active resume not pre-selected; apply refused

BUG-02 · USPS-47 — Major / High

Double approval at once, students notified twice

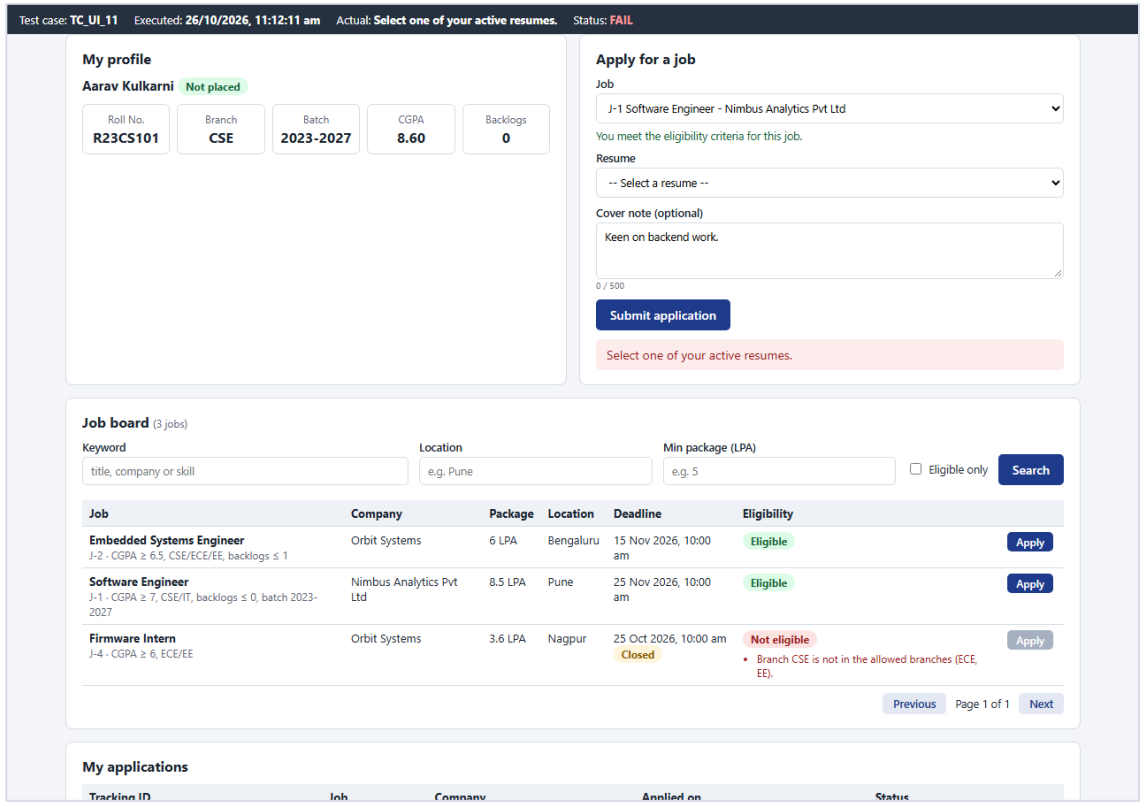
BUG-03 · USPS-48 — Major / High

Same status change applied twice at once

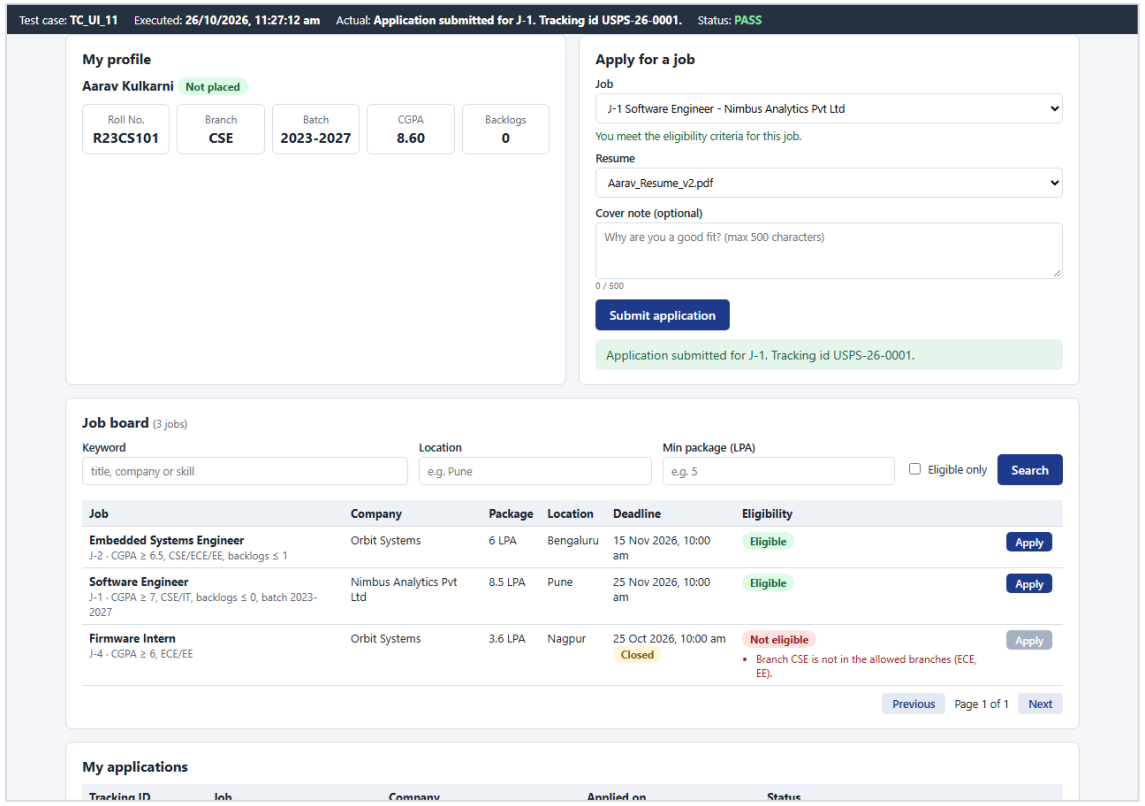
BUG-04 · USPS-49 — Minor / Medium

Posting approved after its deadline

Bug fix and retest evidence · BUG-01



Before fix: TC_UI_11 FAIL (run 1)



After fix: TC_UI_11 PASS (run 2)

Same demo data and test case, captured from the real page in Microsoft Edge before and after the fix

Live demonstration

1

Start the portal

`python run.py  ->  http://127.0.0.1:8091`

2

Recruiter posts, officer approves

two windows: the job appears live, students are notified

3

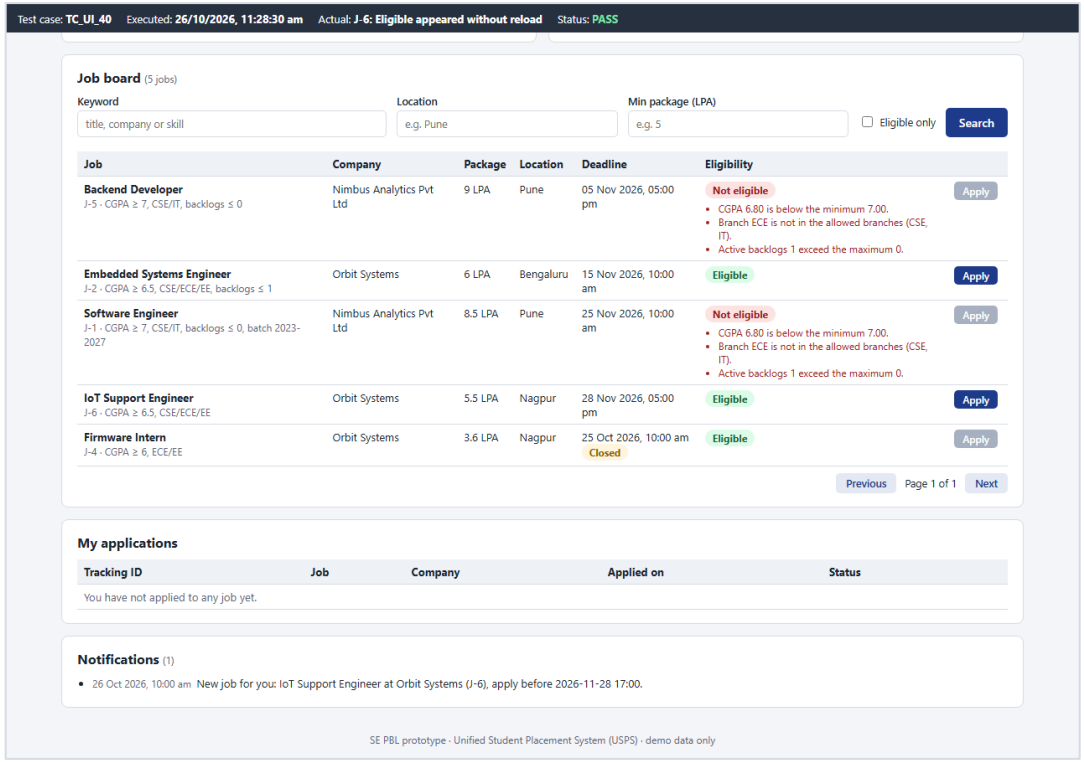
Student applies and is tracked

tracking id, statechart steps, timeline, accept offer

4

Run all tests

`python -m tests.run_tests`



Approved job reaches the eligible student's open job board without a reload

Conclusion and next steps

What was achieved

- Two working modules of my SRS with database, validation, statechart and live pages
- Jira space: 49 work items, 3 sprints, 2 members, bugs tracked to closure
- 155 test cases; 4 bugs found, fixed and retested; 155 / 155 pass

Next steps

- Login and role-based access (SRS 4.1)
- Interview scheduling and offer letters (SRS 4.6, 4.7)
- E-mail notifications and placement reports

Thank you · Questions?